30-Day AI/ML Roadmap (Project-Driven, for Coders)

Who this is for: You can already code comfortably and you're solid with stats/calc. This plan skips "intro to programming" and "intro to algebra" filler and goes straight at ML concepts through projects, using Python's standard data science stack.

Daily time assumption: ~1.5–2 hours/day. Adjust pacing if you have more or less time — the structure matters more than the exact day count.

Philosophy: Learn just enough theory to understand *why* a technique works, then immediately build something with it. Every week ends with a project you can put on GitHub.

Week 1: Foundations — Data Handling + Classical ML Intuition

Milestone: Be fluent in NumPy/Pandas, understand the core ML workflow (train/test split, fit, predict, evaluate), and train your first models.

Day	Task	Resources
1	Set up environment (Python, Jupyter/Colab, NumPy, Pandas, scikit-learn, Matplotlib). Do a NumPy speed-run: arrays, broadcasting, vectorization.	NumPy Quickstart; Google Colab (no setup needed)
2	Pandas speed-run: DataFrames, filtering, groupby, merging, handling missing data.	Pandas 10-min intro
3	Exploratory Data Analysis (EDA): load a real dataset, visualize distributions, correlations, outliers.	Kaggle Titanic dataset; Matplotlib/Seaborn docs
4	Core ML concepts: supervised vs unsupervised, train/test/validation split, overfitting vs underfitting, bias-variance tradeoff.	Google's ML Crash Course – Framing
5	Linear & logistic regression — train your first models in scikit-learn. Understand the math (cost function, gradient descent) conceptually.	StatQuest YouTube: "Linear Regression" & "Logistic Regression"
6	Model evaluation: accuracy, precision/recall, F1, confusion matrix, ROC-AUC, RMSE/MAE for regression.	scikit-learn metrics guide
7	**Project Day.** Build end-to-end: load → clean → EDA → train logistic regression/linear regression → evaluate.	—

Week 1 Project: Titanic survival predictor (classification) or House Prices predictor (regression) — full pipeline from raw CSV to evaluated model.

Week 2: Classical ML Deep Dive — Trees, Ensembles, Feature Engineering

Milestone: Confidently choose and tune models for tabular data; understand feature engineering, which usually matters more than model choice.

Day	Task	Resources
8	Decision trees: how splits work (Gini/entropy), pros/cons.	StatQuest: “Decision Trees”
9	Random Forests & bagging — why ensembles reduce variance.	StatQuest: “Random Forests”
10	Gradient Boosting (XGBoost/LightGBM) — why boosting often wins on tabular data.	XGBoost docs; StatQuest: “Gradient Boost”
11	Feature engineering: encoding categoricals, scaling, handling dates, creating interaction features.	scikit-learn preprocessing guide
12	Cross-validation & hyperparameter tuning (GridSearchCV, RandomizedSearchCV). Avoid overfitting to the test set.	sklearn Cross-validation guide
13	Pick a Kaggle competition/dataset (tabular). Do full EDA + feature engineering pass.	Kaggle Datasets
14	**Project Day.** Train Random Forest + XGBoost, compare against your Week 1 baseline, tune hyperparameters.	—

Week 2 Project: A tabular Kaggle-style competition entry (e.g., House Prices, Customer Churn) with a model comparison writeup: baseline vs. tuned ensemble, with metrics table.

Week 3: Neural Networks & Deep Learning Basics

Milestone: Understand how neural nets actually work (not just “import and call .fit()”), and train your first deep learning models with PyTorch.

Day	Task	Resources
15	Neural network fundamentals: neurons, layers, activation functions, forward pass.	3Blue1Brown: “Neural Networks” series (YouTube)
16	Backpropagation & gradient descent intuition (you have the calc background — lean into this).	3Blue1Brown: Backpropagation
17	PyTorch basics: tensors, autograd, building a simple feedforward net.	PyTorch 60-min Blitz
18	Train a feedforward net on MNIST (digit classification). Understand loss curves, learning rate effects.	PyTorch MNIST tutorial
19	Convolutional Neural Networks (CNNs): why convolution works for images, pooling, common architectures.	StatQuest or CS231n notes
20	Train a CNN on a real image dataset (CIFAR-10 or a custom small dataset).	PyTorch CIFAR-10 tutorial
21	**Project Day.** Build an image classifier end-to-end; try transfer learning with a pretrained model (ResNet) for comparison.	Transfer Learning tutorial – PyTorch

Week 3 Project: Image classifier (e.g., cats vs. dogs, or a dataset relevant to your interests) — trained from scratch

AND fine-tuned via transfer learning, with a comparison of accuracy/training time.

Week 4: Modern AI — NLP/Transformers + Capstone Project

Milestone: Understand the basics of how today's LLMs work, get hands-on with Hugging Face, and ship a polished capstone project.

Day	Task	Resources
22	NLP basics: tokenization, embeddings, why raw text needs vectorization.	Hugging Face NLP Course – Ch.1-2
23	Attention mechanism & Transformers — conceptual deep dive (this is the "how ChatGPT-like models work" day).	Jay Alammar: "The Illustrated Transformer"
24	Hugging Face `transformers` library: load a pretrained model, run inference (sentiment analysis, summarization, etc.).	Hugging Face Quicktour
25	Fine-tune a pretrained transformer on a small custom dataset (e.g., sentiment classification on your own text data).	HF Fine-tuning tutorial
26	Intro to LLM-adjacent tooling: embeddings + vector search (semantic search basics), prompt engineering fundamentals.	HF Sentence Transformers docs
27	Start capstone: pick a project that combines what you've learned (see ideas below). Plan architecture + data.	—
28	Capstone build day 1: data pipeline + baseline model.	—
29	Capstone build day 2: refine model, evaluate, polish.	—
30	**Capstone wrap-up.** Write a clean README, push to GitHub, write a short LinkedIn/blog post explaining what you built and learned.	—

Capstone project ideas (pick one): - A **sentiment/topic classifier** for a domain you care about (reviews, tweets, support tickets), deployed as a simple Streamlit app. - An **image classifier with transfer learning**, wrapped in a small web demo. - A **mini semantic search engine** over a document set you choose, using sentence embeddings. - A **fine-tuned small LLM/transformer** for a specific task (e.g., summarizing articles in a niche you know well).

Resource Summary

- **Courses:** Google ML Crash Course (free, fast); Hugging Face NLP Course (free)
- **Video intuition:** StatQuest (classical ML, stats), 3Blue1Brown (neural net math), Jay Alammar's blog (Transformers)
- **Hands-on docs:** scikit-learn, PyTorch tutorials, Hugging Face docs
- **Datasets/competitions:** Kaggle
- **Compute:** Google Colab (free GPU access — use this if you don't have a GPU locally)

Final Outcome (End of Day 30)

By the end of this roadmap, you will have:

1. **4 real projects on GitHub**, spanning classical ML, ensembles, deep learning (CNNs), and modern NLP/Transformers.
2. **A working understanding** of the ML workflow end-to-end: data prep → feature engineering → model training → evaluation → tuning → deployment-readiness.
3. **Practical fluency** with the tools used in real ML jobs: Pandas, scikit-learn, PyTorch, Hugging Face.
4. **A capstone project** that demonstrates applied skill — strong enough to talk through in an interview or showcase in a portfolio.
5. **A clear sense of direction** for where to go next (e.g., deeper into deep learning, MLOps/deployment, or a specific domain like CV or NLP).

Note on pacing: If a day's topic doesn't click in one sitting, don't be afraid to spend 2 days on it and compress a lighter day elsewhere — the weekly *projects* are the real checkpoints, not the daily schedule.